

Home Assistant Add-on Custom UI — Project Log

Overview

This project aims to build a scalable, modular, professional-grade user interface for a Home add-on, inspired by high-end smart home dashboards. The UI must support responsive layouts phone, tablet, and desktop, and allow end users to customize widgets dynamically.

Goals

Primary Goals

- Create a modern dashboard UI
- Support widget-based layout
- Allow drag, resize, reorder
- Work smoothly on mobile + tablet
- Be scalable long-term
- Separate UI logic from data logic

Secondary Goals

- Theme support (dark/light)
 - Smooth animations
 - Modular architecture
 - Plugin-style widget system
-

Architectural Decisions

Framework Choice

Selected: **React + Vite**

Reason: - component architecture - scalable - ecosystem support - routing support - maintainability

Rejected options: - vanilla JS → not scalable enough - full framework (Angular/Vue full stack) unnecessary complexity

Layout Engine

Chosen library: **react-grid-layout**

Reasons: - drag & drop built-in - resize support - responsive breakpoints - layout persistence
production-proven

Layout will be generated dynamically from config JSON.

Example layout schema:

```
{
  "layout": [
    { "type": "solar", "x":0, "y":0, "w":2, "h":1 },
    { "type": "climate", "x":2, "y":0, "w":1, "h":1 }
  ]
}
```

Widget System Design

Widgets are independent components.

Folder structure:

```
/widgets
  SolarWidget.jsx
  ClimateWidget.jsx
  SecurityWidget.jsx
```

Widget registry pattern:

```
export const widgetRegistry = {
  solar: SolarWidget,
  climate: ClimateWidget,
  security: SecurityWidget
};
```

Dynamic rendering:

```
const Widget = widgetRegistry[item.type];
return <Widget {...item.props} />;
```

Data Layer

UI must not call Home Assistant directly.

Architecture:

UI → Add-on backend proxy → Home Assistant API

Reasons: - avoids CORS - hides tokens - improves security - centralizes API logic

Backend options: - Node (Express) OR - Python (FastAPI)

Final selection pending.

Responsiveness Strategy

Not CSS-only responsiveness.

Use breakpoint layouts:

```
lg  
md  
sm  
xs
```

Each breakpoint has its own layout config.

State Management

Planned: - Zustand or Redux Toolkit

Purpose: - global entity states - widget config - theme state - connection status

Persistence

User layout + settings stored in:

Phase 1 - localStorage

Phase 2 - backend database / JSON store

UI Structure

Main Sections

- Home dashboard
- Rooms
- Energy
- Security
- Settings

Navigation type: - bottom nav (mobile) - sidebar (tablet/desktop)

Widget Types (Initial)

Core widgets: - Power flow - Climate - Lights summary - Security state - Battery - Weather

Future widgets: - camera preview - automation triggers - media control - notifications

Development Phases

Phase 1 — Foundation

- project scaffold
- routing
- layout engine
- widget loader

Phase 2 — Core Widgets

- implement first 5 widgets
- realtime data connection

Phase 3 — Customization

- drag
- resize
- add/remove widget

Phase 4 — Persistence

- save layout
- load layout

Phase 5 — Advanced UX

- animations
- transitions
- gesture support

Phase 6 — Plugin System

- external widget loading
 - widget marketplace structure
-

Styling Strategy

Design principles: - large spacing - rounded panels - subtle shadows - minimal borders - soft gradients

UI tokens: - border radius scale - spacing scale - elevation scale - color roles

Risks

Technical risks: - websocket performance - layout reflow lag - mobile performance

Mitigation: - memoization - virtualization - throttled updates

Anti-Patterns to Avoid

Do NOT: - hardcode widget placement - fetch HA directly from frontend - couple UI to engine
layout only in memory

Long-Term Vision

Target outcome: A modular smart-home dashboard platform that can: - scale to dozens of vendors
support plugins - support themes - support multiple user profiles

Final state = framework, not UI.

Next Steps Checklist

- [] choose backend language
- [] create repo

- [] scaffold Vite React app
 - [] install grid layout
 - [] implement widget registry
 - [] create base theme
 - [] create first widget
-

Project Philosophy

Build infrastructure first. UI second. Features last.

Reason: Architecture determines scalability.

END OF LOG